ERA · AI CLINIC ASSISTANT

# Era — Simple Guide

Features & How Everything Works — in simple language

Version 4.3 · October 5, 2026 · for clinic staff and non-technical readers

# Contents


1. Era — Simple Guide: Features & How Everything Works
    - The big picture (30 seconds)
    - 1. Signing in — who can use Era
    - 2. Asking questions — the chat brain
    - 3. Clinic data — appointments, patients, prescriptions
    - 4. The voice — Sofia
    - 5. The screen — what you see
    - 6. Setup, safety and hosting
    - Flow 1 — When the server starts
    - Flow 2 — Signing in (three ways)
    - Flow 3 — The pop-up greeting
    - Flow 4 — Asking a question (the main loop — the heart of Era)
    - Flow 5 — Health questions
    - Flow 6 — Booking, moving, cancelling (by voice)
    - Flow 7 — Admin and Front Desk
    - Flow 8 — Connecting a real hospital system (RMS)
    - Flow 9 — When things go wrong (and what happens)
    - Flow 10 — Checking and publishing

# Era — Simple Guide: Features & How Everything Works

**Era is a friendly AI assistant for a doctor's clinic.** She appears on the screen as a character, listens to your questions (by voice or typing), and **speaks the answers out loud** in one voice — Sofia (English or Hindi, chosen automatically).

This guide explains everything in **simple words**:

1. **Part 1** — every feature, and how it is built.
2. **Part 2** — exactly how everything works, step by step.

*(Version 4.3 · October 2026)*

### The big picture (30 seconds)

- The app has two pieces: a **web page** (what you see) and a **small server** (the brain that answers). No extra software to install.
- All answers come from the **server**, never from the web page. This keeps security in one place.
- The server uses a **database file** (SQLite — a small, reliable database that is built into Node, like a tidy Excel file but safer).
- Speaking is done by **NVIDIA's cloud voice "Sofia"**. The secret API key stays on the server only.
- Login uses a **secure pass** (a signed token, valid 8 hours) — like a wristband at an event that proves you already signed in.

---

# Part 1 — Features list (and how each is built)

### 1. Signing in — who can use Era

- **One-click demo login.** Pick a name — Dr. Amit, Dr. Rahul, Dr. Priya, Admin, or Front Desk — and you are in, no password. Built so anyone can try the app instantly. There is a switch (`DEMO_LOGIN`) to turn this off for real use; when off, the door simply disappears.
- **Email sign-in that also registers.** One form does both jobs. Existing email + right password → you are in. New email → the server **creates the account automatically** (role: doctor) and signs you in. Wrong password → a helpful message.
- **Passwords are never stored as text.** They are scrambled with a strong one-way math function (scrypt) before saving. Even the demo password `password123` is stored scrambled.
- **The login pass (JWT).** After signing in, the server gives the browser a signed pass valid for 8 hours. The browser keeps it in a protected cookie and shows it on every request. No pass → no data.
- **Three roles.** `doctor` sees only their own data. `admin` may deliberately view another doctor's data. `reception` gets health questions and today's clinic schedule only.
- **Stay signed in.** Reopen the page and, if the pass is still valid, you land straight in the workspace — no re-typing.
- **Sign out.** One click deletes the pass (and stops any speech).

## 2. Asking questions — the chat brain

- **One door for all answers.** Every question is sent to `POST /api/assistant/chat` on the server. The web page itself never invents answers, so rules can't be bypassed from the browser.
- **Understanding the question.** The brain (a function called `routeIntent`) reads the words and picks an action: "next appointment", "cancel 12", "book…", "meetings", "appointments tomorrow", "free slots", "my patients", "my prescriptions", "my profile", or a health question.
- **Health questions.** 15 ready-made answers live in the database (fever, cold, headache, blood pressure, diabetes, sleep, diet, stress…). The server scores every row by word matching and speaks the best one. Urgent topics always include "see a doctor".
- **You only see your own data.** Every database read is automatically locked to your doctor ID. Another doctor's rows simply don't exist for you.
- **Nothing private leaks.** Internal IDs (doctor_id, patient_id) are stripped from replies before they leave the server.
- **Everything is logged.** Every chat, tool call, login, and denial is written to an audit log (who, what, when, request ID) — but never the message text itself.
- **Polite errors.** Not signed in → "Please sign in…". Asking for someone else's data → a friendly "I can only access information available to your RMS account." No scary codes, no technical details.

## 3. Clinic data — appointments, patients, prescriptions

- **A real database with nothing to install.** SQLite comes built into Node (version 22.5+). One file holds everything.
- **Appointments.** List for today / tomorrow / any day, see details, book new, move, or cancel. **Double-booking is blocked** — two appointments can't take the same time slot.
- **Free-slot finder.** Reads the doctor's weekly working hours, removes booked times, and shows the free 30-minute slots.
- **Patients and prescriptions.** Each doctor sees only patients linked to them and prescriptions they issued.
- **Self-freshening demo data.** At every server start, the sample appointments are re-dated to **today and tomorrow**, so "my appointments tomorrow" never shows an empty list in demos.
- **Safe in the cloud.** If the database folder is missing or read-only (happens on Render's free hosting), the app tries other locations (project folder → system temp folder) instead of crashing, and rebuilds everything automatically.

## 4. The voice — Sofia

- **One voice only.** NVIDIA's **Sofia** — English and Hindi. (This was a deliberate choice: a single, natural female voice.)
- **Automatic language.** If the reply contains Hindi or Hinglish words ("namaste", "bukhar"…), Sofia speaks Hindi; otherwise English. No language button needed.
- **The secret key stays secret.** The NVIDIA API key lives only on the server (in an environment variable or a git-ignored file). It is never sent to the browser and never logged.
- **Survives a moody voice service.** NVIDIA sometimes fails for a few minutes. Era tries **3 times over the web**, then **2 times through a backup path** (a small Python bridge using gRPC). Each answer is checked to be real audio before use.
- **Instant repeats.** The last 20 spoken answers are kept in memory — the same question's answer plays instantly.

- **No fake fallback.** If the voice service is down, Era shows "Voice service unavailable — please try again shortly." She never switches to the browser's robotic voice — that was removed on purpose.
- **Stop anytime.** The Stop button, a new question, or sign-out cuts the sound instantly (a "generation token" cancels playback).
- **Listening.** The mic uses the browser's speech recognition (Chrome/Edge). If the browser can't listen, a slim typing box is always there.

### 5. The screen — what you see

- **Two screens only.** The login card, then the workspace. Nothing else to learn.
- **The character.** A 3D-style photo that gently floats and "breathes"; wave bars animate while she speaks; a status line shows Ready / Listening… / Speaking… / Thinking….
- **Pop-up greeting.** On sign-in she says "Good morning, Dr. Amit! I'm Era, your assistant…" — **once per login session**; refreshing the page stays quiet.
- **Kind to screen readers.** Every reply is also placed in a hidden area (aria-live) so it is readable when sound is off or unavailable.
- **Settings dialog.** Your name, "Era speaks replies" on/off, "Greet me" on/off, and the voice (Sofia · English / Sofia · हिन्दी). Saved in the browser.
- **Always fresh.** The app files are served with "no-cache", so an old broken version never sticks in the browser.

### 6. Setup, safety and hosting

- **One config file.** `.env` (git-ignored) holds port numbers, the login secret, the NVIDIA key, and switches. The file always wins over messy computer-wide settings (which once broke the launch with a junk `PORT=0`).
- **Optional HTTPS.** If certificate files exist, a secure local address (port 8443) is served too.
- **Auto-restart watchdog.** `start-era.cmd` restarts the server every 3 seconds if it dies.
- **18 automatic tests.** A security test suite (separate scratch database) checks login, roles, data isolation, audit and sanitisation before anything ships.
- **Runs on Render (free tier).** Build = run tests, start = run server. It sleeps after ~15 minutes idle and wakes in 30–50 seconds; the database re-creates itself if the host wipes it.

---

# Part 2 — Full working flow, step by step

### Flow 1 — When the server starts

1. The server reads the `.env` config file (ports, secrets, switches).
2. It opens the database file — trying your configured path first, then the project folder, then the system temp folder (so it never crash-loops on strict hosting).
3. It creates any missing tables (users, doctors, patients, appointments, prescriptions, audit log, knowledge base, schedule) and adds missing columns to older databases.
4. It fills the demo clinic: 3 doctors, Admin, Front Desk, 5 patients, weekly hours, sample appointments (dated today/tomorrow), 2 sample prescriptions, and the 15 health answers.
5. It starts the web server on port 8000 (and 8443 with HTTPS if certificates exist). Now the app works at `http://localhost:8000`.

## Flow 2 — Signing in (three ways)

- **Demo:** choose a name → the server finds that demo user → creates the signed pass → straight to the workspace.
- **Email:** type email + password → checked against the scrambled password → in. New email → account created first, then in.
- **Already signed in:** the page asks the server "who am I?" and, if the pass is valid, skips the login screen.

## Flow 3 — The pop-up greeting

Once per login session (a small flag in the browser's session storage tracks this), Era builds a greeting from the clock and your name — "Good morning, Dr. Amit!…" — and speaks it after a short pause. Refreshing the page does not repeat it. Signing out clears the flag, so your next login greets you again.

## Flow 4 — Asking a question (the main loop — the heart of Era)

1. **You press 🎤 Talk.** The browser listens and turns your speech into text.
2. **The text travels to the server** (`POST /api/assistant/chat`) together with your login pass (cookie).
3. **The server checks the pass.** Invalid or expired → "Your session expired — please sign in again." and you return to the login screen.
4. **The brain reads the text** and picks the best action: next appointment, cancel #12, book, meetings, appointments (today/tomorrow), free slots, working hours, patients, prescriptions, profile, or a health question. First match wins.
5. **The action queries the database** — every query is locked to *your* doctor ID inside the SQL itself.
6. **Safety cleaning:** internal IDs are removed, and the raw rows are turned into a friendly sentence — "You have 2 appointments: tomorrow, 10:00 AM to 10:30 AM — BP follow-up…".
7. **The action is written to the audit log** (user, doctor, action, request ID — never the message text).
8. **The sentence returns to the browser.** It appears in the hidden screen-reader area and the status changes to "Speaking…".
9. **The voice pipeline starts:** Is the answer already in the 20-item memory cache? → play instantly. Not cached? → ask NVIDIA (up to 3 tries) → if still nothing, try the backup path (2 tries) → check the audio is genuine → store in cache → play through the audio player.
10. **If every voice attempt fails**, the status shows "Voice service unavailable — please try again shortly" and the speaker stays silent — but the text is still there for screen readers.

## Flow 5 — Health questions

"i have a fever" matches no clinic action → the server scores all 15 health rows → the fever answer wins → spoken in Sofia, and urgent topics always add "see a doctor". "what is the weather" matches nothing → Era honestly says what she *can* help with, instead of guessing.

## Flow 6 — Booking, moving, cancelling (by voice)

1. "add a meeting called Review at 3 pm" or "cancel appointment 12" → the brain detects a **write** action.
2. The tool checks: is the patient linked to you? is the slot actually free?
3. All good → the change is saved; a problem → a clear spoken answer ("That time slot is already booked.").

4. Success is spoken too — "Booked your appointment (#13)." — and written to the audit log with the record number.

## Flow 7 — Admin and Front Desk

They have no personal doctor data, so schedule-style questions are answered from the **shared clinic schedule** ("3 meetings and 2 appointments today…"). If they ask for doctor-only data, they get a polite denial — the security layer still enforces it underneath. An **admin** can deliberately view a specific doctor's data by naming the doctor ID; the server validates it.

## Flow 8 — Connecting a real hospital system (RMS)

The same endpoints are dashboard-ready: your hospital's login can replace Era's (one function), the demo data can be removed, and the demo door switched off. The web page or your dashboard talks over normal REST calls (appointments, patients, prescriptions) with the same login pass and the same "only your data" rule.

## Flow 9 — When things go wrong (and what happens)

| What happens | What Era does |
|---|---|
| NVIDIA voice service is down or slow | Retries 3× web + 2× backup path; then shows "Voice service unavailable" — text replies still visible |
| No NVIDIA key on the server | Clear "not configured" message; nothing crashes |
| Someone asks for a different voice | Blocked — only Sofia (English/Hindi) is allowed |
| Database folder missing (strict hosting) | Tries other folders; re-creates and re-seeds the database |
| Server unreachable | "I cannot reach the clinic server right now. Please check the connection and try again." |
| Login pass expired | "Your session expired — please sign in again." + back to login screen |
| Browser cannot listen (no mic support) | Spoken hint + the typing box |
| Same slot booked twice | "That time slot is already booked." |

## Flow 10 — Checking and publishing

- `npm test` runs the 18 security checks on a throwaway database (the real data is never touched).
- `node server.js` starts everything locally; `start-era.cmd` keeps it alive forever (auto-restart every 3 s).
- On Render: build = run the tests, start = run the server; put the NVIDIA key, login secret and settings in the dashboard; the database re-seeds itself after each redeploy.

---

*That's the whole system: one page, one small server, one database file, one voice — with doctor-level privacy enforced in the database itself.*